TraceBack — Deployment Documentation

Nov 6, 2026 · @Soumyaranjan

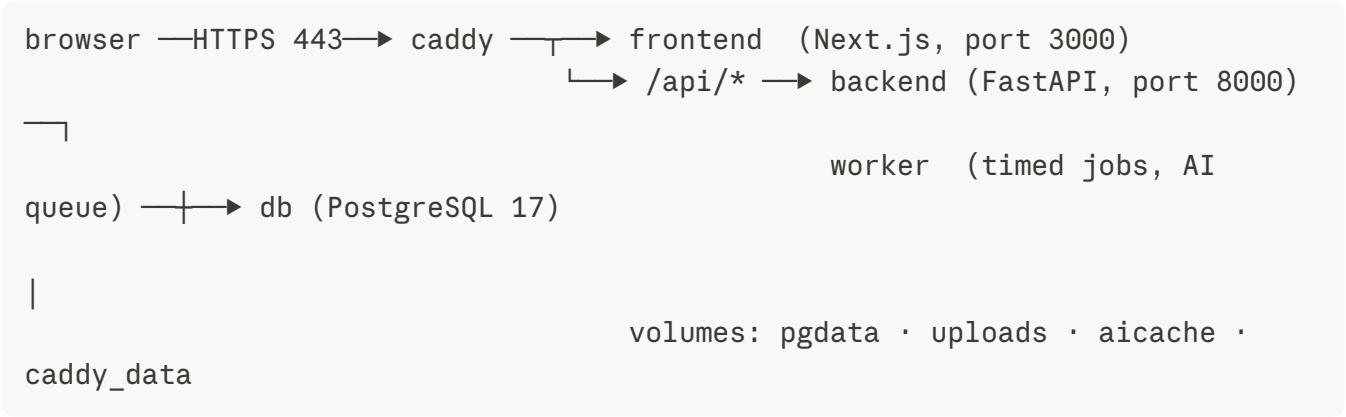
TraceBack runs on one AWS EC2 t3.small server in Sydney as five Docker containers behind Caddy, which serves it over HTTPS at https://16-176-186-149.sslip.io. A release is one command from a team laptop (`deploy\upload.ps1`): it uploads the code, rebuilds the containers and applies database migrations, with data kept in Docker volumes across releases. The first deployment was made on 3 Nov 2026.

TraceBack is the IIT Kanpur Lost & Found portal. Everything needed to deploy it is in the repository's `deploy/` folder; this document explains it for whoever runs the server.

1. At a glance

Item	Value
Address	https://16-176-186-149.sslip.io (sslip.io maps the name to the server's IP, so HTTPS works without buying a domain)
Cloud	AWS, region Asia Pacific (Sydney) `ap-southeast-2`; account on the Free plan
Server	EC2 `t3.small`: 2 vCPU, 2 GB RAM, plus 2 GB swap; 20 GB gp3 disk; Elastic IP 16.176.186.149
Operating system	Ubuntu Server 24.04 LTS
Runtime	Docker with Docker Compose v2
HTTPS	Caddy 2, automatic Let's Encrypt certificate, HTTP redirected to HTTPS
Deploy from	Windows laptop with PowerShell and OpenSSH (`upload.ps1`)
Repository	github.com/cs455project/campus-lost-and-found, folder `deploy/`

2. Architecture



Only Caddy has public ports. The database, the API and the website are reachable only inside the Docker network.

Container	Image / build	Role	Health check and restart	Data
`caddy`	`caddy:2`	HTTPS, security headers, `/api/*` to the backend, the rest to the website	`restart: unless-stopped`	`caddy_data` (certificates)
`frontend`	`frontend/Dockerfile` (Node 22, `next build`, `next start`)	The website	`restart: unless-stopped`	none
`backend`	`backend/Dockerfile` (Python 3.14)	FastAPI with 2 uvicorn workers; runs `alembic upgrade head` and the seed on every start	`/health` every 10 s; `restart: unless-stopped`	`uploads` (photos, proofs), `aicache` (AI model)
`worker`	same image as the backend	Reminders, expiry, clean-up, the AI queue	starts after the backend is healthy; `restart: unless-stopped`	`uploads`, `aicache`

Container	Image / build	Role	Health check and restart	Data
`db`	`postgres:17`	The database	`pg_isready` every 5 s; `restart: unless-stopped`	`pgdata`

3. Configuration

Settings live in `deploy/.env` on the server only (mode 600, never committed). `deploy.sh` creates it on the first run from `deploy/.env.example`, filling in the address and random values for the database password and `SECRET_KEY`. The email and AI keys are copied from a team laptop with `deploy\set-secrets.ps1`, which prints nothing.

Setting	Production value	Why
`SITE_ADDRESS`, `FRONTEND_ORIGIN`	`16-176-186-149.sslip.io`, `https://…`	Caddy's certificate; the only origin the API accepts
`DB_PASSWORD`, `SECRET_KEY`	random, generated on the server	Database login; signs sessions and hashes codes
`COOKIE_SECURE`	`true`	Session cookie only over HTTPS
`EMAIL_BACKEND`, `SMTP_*`	`smtp`, Gmail with an app password	Sign-in codes and notices
`LLM_API_KEY`, `GEMINI_API_KEY`	free-tier keys	Groq (`openai/gpt-oss-20b`) for the agent; Gemini (`gemini-2.5-flash`) as fallback and for photo descriptions
`AGENT_MATCHING`, `AI_DAILY_CALL_LIMIT`	`ambiguous`, 500	Call the AI only when the maths is unsure; never more than 500 calls a day
`API_DOCS`	`false` (set in `docker-compose.yml`)	No public `/docs` or `/openapi.json`

Setting	Production value	Why
`AI_WORKER_IN_API`	`false` (set in `docker-compose.yml`)	The AI queue runs only in the worker

4. First-time setup

1. **Launch the server** (AWS console): Ubuntu 24.04, `t3.small`, a new RSA key pair (`.pem`), 20 GB gp3, and a security group allowing SSH from *My IP* and HTTP/HTTPS from anywhere. Allocate an Elastic IP and associate it, so the address survives a stop and start.
2. **Protect the key** on Windows: `icacls .\lostfound-key.pem /inheritance:r /grant:r "$($env:USERNAME):R"`.
3. **Deploy**, from the project folder: `powershell -ExecutionPolicy Bypass -File deploy\upload.ps1 -Server 16.176.186.149 -Key C:\path\to\lostfound-key.pem`. The first run installs Docker, adds 2 GB of swap, creates `deploy/.env`, builds everything (5–10 minutes) and prints the address.
4. **Add the secrets and the admin**: `powershell -ExecutionPolicy Bypass -File deploy\set-secrets.ps1 -Server 16.176.186.149 -Key C:\path\to\lostfound-key.pem -AdminEmail <admin email>`.
5. **Set the offices**: on the server, `sudo docker compose exec backend python -m app.cli set-office "Main Security Office" --where "<directions>" --hours "24 hours"` for each office.

5. Releasing an update

`upload.ps1` packs `backend/`, `frontend/` and `deploy/` (never `.env` files, uploads, `node_modules` or caches), copies them to the server and runs `deploy.sh` in the background, so the build continues if the SSH connection drops (`tail -f ~/deploy.log` to watch). `deploy.sh` rebuilds the images and restarts the containers; the backend applies new migrations before it starts serving. Data in the volumes is untouched.

- **Downtime:** about 1–2 minutes while the new containers start.
- **Rollback:** check out the previous release on the laptop and run `upload.ps1` again. Migrations in this release only add a column, so the older code runs on the newer schema; `alembic downgrade -1` removes it if needed.

6. Operations

On the server, in `~/lost-found/deploy`:

Task	Command
What is running, and healthy	`sudo docker compose ps`
Follow the logs	`sudo docker compose logs -f backend worker`
Memory and CPU per container	`sudo docker stats`
Restart everything	`sudo docker compose restart`
Stop (data kept)	`sudo docker compose down`
Back up the database	`sudo docker compose exec -T db pg_dump -U lostfound lostfound > backup-$(date +%F).sql`
Restore the database	`sudo docker compose exec -T db psql -U lostfound lostfound < backup.sql`
Back up photos and proofs	`sudo docker run --rm -v deploy_uploads:/data -v $PWD:/b alpine tar czf /b/uploads.tgz -C /data .`
Make an admin	`sudo docker compose exec backend python -m app.cli make-admin <email>`
Check email delivery	`sudo docker compose exec backend python -m app.cli email-test <address>` and `… email-log`
Check the AI keys	`sudo docker compose exec backend python -m app.cli ai-check`

Backups contain personal data: keep them on the server or encrypted, never in the repository (`backup*.sql` is git-ignored).

7. Security settings

- **Network:** the security group opens 443 and 80 (redirected to HTTPS) to everyone and SSH (22) only to the team's current IP ("My IP"), never 0.0.0.0/0; the database and API have no public port.
- **Access:** SSH with the key file only (no passwords); MFA on the AWS account and 2-step verification on the portal's and the staff's Gmail accounts.

- **HTTPS and headers:** Caddy adds HSTS, `nosniff`, `X-Frame-Options: DENY` and `Referrer-Policy`, and hides `Server` and `X-Powered-By`; the website and the API set the same headers themselves.
- **Secrets:** only in `deploy/.env` on the server; rotate the AI keys and the Gmail app password if they are ever exposed, and change `SECRET_KEY` to sign everyone out.
- **Requests:** uploads up to 25 MB per request (photos 5 MB each), at most 30 requests in flight per API process.

8. Checks after every deployment

1. `sudo docker compose ps`: all 5 containers up, `backend` and `db` healthy.
2. Open the address: the home page loads over HTTPS with a valid certificate.
3. `curl -s https://16-176-186-149.sslip.io/api/health` returns `{"status":"ok","database":"ok"}`.
4. `curl -sI https://16-176-186-149.sslip.io` shows `strict-transport-security`, `x-content-type-options` and `x-frame-options`.
5. `https://16-176-186-149.sslip.io/api/docs` returns 404.
6. Sign in with an @iitk.ac.in address: the code arrives by email.
7. `sudo docker compose logs worker | tail` shows the worker started with no errors.
8. Once per release, `sudo reboot`; after 2 minutes all containers are back by themselves.

9. Troubleshooting

Symptom	Cause and fix
`ssh: connect … port 22: Connection timed out`	Your IP changed (mobile data) or the network blocks SSH; set the SSH rule to *My IP* again
`client_loop: send disconnect` during a deploy	The build continues on the server; watch `tail -n 40 -f ~/deploy.log`
The site doesn't load	Check that ports 80 and 443 are open, then `sudo docker compose logs caddy`
"Can't reach the server" on the site	`sudo docker compose logs backend`, often a typo in `.env`

Symptom	Cause and fix
“TraceBack is briefly unavailable”	The database is restarting or down: `sudo docker compose ps db` and `logs db`
Build killed, out of memory	Keep the instance at `t3.small` or larger; the 2 GB swap must be on (`swapon --show`)
No sign-in emails	`… app.cli email-log` ; check `SMTP_PASSWORD` is a Gmail app password

10. Costs

The server costs about US$25 a month at on-demand prices and is paid from the AWS Free plan credits; see the Costing tab.